

Persistencia de la Quantum Wallet con SQLite

Del modelo de clases UML al esquema relacional de Quantum Core

Diseño e implementación de una base de datos relacional con llaves primarias y foráneas, tipos de datos coherentes y restricciones de integridad, generada de forma automatizada desde Python, con un proceso de carga masiva de registros desde Excel

ESTUDIANTE	Deibis Zuluaga Baena
NÚCLEO	Fundamentos de Software · FUNSO_2_N_H
ACTIVIDAD	Introducción a las bases de datos
HERRAMIENTAS	SQLite 3 · Python 3.14 (sqlite3, openpyxl) · DB Browser for SQLite · Microsoft Excel
ENTREGABLES	configurar_db.py, quantum_wallet.db, generar_excel.py, importar_excel.py, modelo entidad-relación (PNG, SVG y .drawio) e informe
DOCENTE	Simón Peláez Loaiza
INSTITUCIÓN	CEIPA Business School
FECHA DE ENTREGA	4 de octubre de 2026

24 páginas · 18 figuras

Objetivo del trabajo

Objetivo: transformar el modelo de clases de la Quantum Wallet (`Usuario` , `UsuarioEmpresa` y `wallet`) en un esquema de base de datos relacional en SQLite que garantice la persistencia de la información de Quantum Core, mediante tablas con llaves primarias y foráneas, tipos de datos coherentes y restricciones de integridad, y automatizar su creación con el script `configurar_db.py` .

CÓDIGO ORIGINAL DE LA ACTIVIDAD

El enunciado de la actividad y el repositorio del curso definen dos tablas: `usuarios` , con nombre, correo electrónico y NIT, y `wallets` , con el saldo y la llave foránea `id_propietario` , junto con una inserción de prueba. Ese diseño se conserva completo. Para enriquecer el ejercicio, el esquema se amplió hasta cubrir todas las clases del modelo UML de la Quantum Wallet, incluidas `Transaccion` y `Empleado` , la agregación entre empresas y empleados, y la configuración global del Banco Central. Los elementos adicionales se identifican en el código con el comentario `[AGREGADO]` y en el modelo entidad-relación con fondo amarillo.

ALCANCE DEL ESQUEMA

La base de datos resultante está compuesta por seis tablas, cuatro triggers, tres vistas y dos índices. Además de almacenar la información, el esquema traslada a la base de datos las reglas del modelo de clases: la composición entre el usuario y su billetera, la herencia de los empleados, la agregación entre empresas y empleados, el límite por transacción y la instancia única de la configuración global. Como mejora, se incorporó un proceso de carga masiva que importa desde Excel 560 registros simulados y somete cada fila a esas mismas reglas.

REQUISITOS DEL ENUNCIADO

El enunciado plantea transformar las clases en tablas, definir sus campos, asignar tipos de datos coherentes, establecer la llave foránea entre billeteras y usuarios, automatizar la creación de la base con `configurar_db.py` , insertar datos de prueba y verificar el resultado en un visor de bases de datos. Las tablas se identifican en la sección 03; los campos y sus tipos, en la sección 05; la llave foránea `id_propietario` y las demás relaciones, en la sección 06; el script y los datos de prueba, en las secciones 07 y 08, y la verificación con DB Browser for SQLite, herramienta que cumple la función del visor indicado en el enunciado, en la sección 09.

CRITERIO PARA LOS TIPOS DE DATOS

SQLite almacena los valores en clases de almacenamiento. Se asignó `INTEGER` a los identificadores; `REAL` a los valores monetarios (saldo, monto, salario y límite), con el fin de conservar los centavos; y `TEXT` a nombres, documentos, códigos y fechas. La cédula y el NIT se definieron como texto porque son identificadores y no cantidades: pueden contener guiones o ceros a la izquierda. Las fechas se registran en formato ISO 8601 (`AAAA-MM-DD HH:MM:SS`), que SQLite ordena y compara de forma correcta.

Instalación y verificación de SQLite

La persistencia se implementó con SQLite, un motor de base de datos relacional que almacena la base completa en un único archivo y no requiere un servidor. El entorno se preparó en macOS con los componentes que se describen a continuación.

COMPONENTE	VERSIÓN	FUNCIÓN EN EL PROYECTO
DB Browser for SQLite	Aplicación de escritorio	Herramienta gráfica de código abierto instalada para administrar SQLite. Se utilizó para abrir <code>quantum_wallet.db</code> e inspeccionar su estructura y sus datos.
Consola <code>sqlite3</code>	SQLite 3.43.2	Intérprete de línea de comandos incluido en macOS, ubicado en <code>/usr/bin/sqlite3</code> .
Python y módulo <code>sqlite3</code>	Python 3.14.7 · SQLite 3.50.4	Biblioteca estándar de Python con la que <code>configurar_db.py</code> crea la base, inserta los datos y ejecuta las verificaciones.
Librería <code>openpyxl</code>	3.1.5	Lectura y escritura de archivos de Excel (<code>.xlsx</code>) desde Python, utilizada en la carga masiva.
Microsoft Excel	—	Revisión del archivo de carga y del reporte de registros rechazados.
Visual Studio Code	—	Editor del código fuente y del modelo entidad-relación, este último con la extensión Draw.io Integration.

Verificación de la instalación

La instalación se comprobó desde la Terminal con la ubicación y la versión de la consola `sqlite3`, la versión de Python y la importación del módulo `sqlite3`, que imprime la versión de la biblioteca con la que trabaja el script.



```

zulu — -zsh — 147x48
/usr/bin/sqlite3
3.43.2 2023-10-10 13:08:14 1b37c146ee9ebb7acd0160c0ab1fd11017a419fa8a3187386ed8cb32b709aapl (64-bit)
Python 3.14.7
SQLite listo para Python - 3.50.4
zulu@MacBook-Air-de-Deibis ~ %

```

Figura 1. Verificación de SQLite en la Terminal de macOS: consola `sqlite3` 3.43.2, Python 3.14.7 y módulo `sqlite3` con SQLite 3.50.4.

VERSIONES DEL MOTOR

La consola y Python reportan versiones distintas porque cada uno incorpora su propia copia de la biblioteca SQLite. Ambas comparten el mismo formato de archivo, por lo que la base creada desde Python se abre sin cambios desde la consola y desde DB Browser for SQLite. Python no requiere instalaciones adicionales: el módulo `sqlite3` forma parte de su biblioteca estándar.

Identificación de las entidades

Cada clase del modelo UML con identidad y datos propios se convirtió en una tabla. Las relaciones de herencia, composición y agregación se resolvieron mediante llaves foráneas y restricciones.

CLASE	TABLA	ORIGEN	CRITERIO DE DISEÑO
Usuario UsuarioEmpresa	usuarios	Original	Ambas clases comparten identidad y atributos; la subclase solo agrega el NIT. Se almacenan en una tabla con la columna tipo_usuario , lo que evita duplicar datos y uniones innecesarias.
Wallet	wallets	Original	Concentra el dinero del usuario y se vincula a su propietario mediante id_propietario .
Transaccion	transacciones	Adicionada	El historial es una lista dentro de wallet . En el modelo relacional, una lista se representa como una tabla con una fila por movimiento.
Empleado	empleados	Adicionada	Agrega cargo y salario. Se almacena en una tabla propia cuya llave primaria es, a la vez, llave foránea a usuarios , para no dejar columnas vacías en los demás tipos de usuario.
UsuarioEmpresa ◇ Empleado	vinculaciones	Adicionada	La agregación es de muchos a muchos: una empresa agrupa varios empleados y un empleado puede vincularse a varias empresas. Esa relación requiere una tabla intermedia.
BancoCentral	configuracion_sistema	Adicionada	Los parámetros globales (moneda y límite por transacción) se leen desde la base. La tabla admite una sola fila, del mismo modo que el Singleton admite una sola instancia.

Decisiones de diseño

DECISIÓN	JUSTIFICACIÓN
Herencia resuelta en dos formas	Usuario y UsuarioEmpresa comparten la tabla usuarios porque la diferencia es una sola columna. Empleado se separa porque agrega dos atributos obligatorios que no aplican a los demás usuarios. En ambos casos se evitan columnas vacías y datos repetidos.
Identificador de la billetera	En la clase, id_billetera se genera con id(self) , una dirección de memoria que cambia en cada ejecución y no sirve como dato persistente. Se reemplazó por un código estable (W-00001) derivado de la llave primaria.
Contador de transacciones	El atributo estático _contador de Transaccion se reemplazó por AUTOINCREMENT : la base asigna el número y lo conserva entre ejecuciones.
Listas de objetos	Las listas historial y empleados se reemplazaron por tablas relacionadas mediante llaves foráneas.
Métodos de las clases	Los métodos no se almacenan. Los que modifican datos (recargar() , realizar_pago() , transferir() y pagar_nomina()) se implementaron como funciones de configurar_db.py que ejecutan las sentencias SQL dentro de una transacción.

Esquema relacional de quantum_wallet.db

Quantum Wallet · Modelo entidad-relación

Esquema relacional de quantum_wallet.db (SQLite) derivado del diagrama de clases UML de usuarios.py

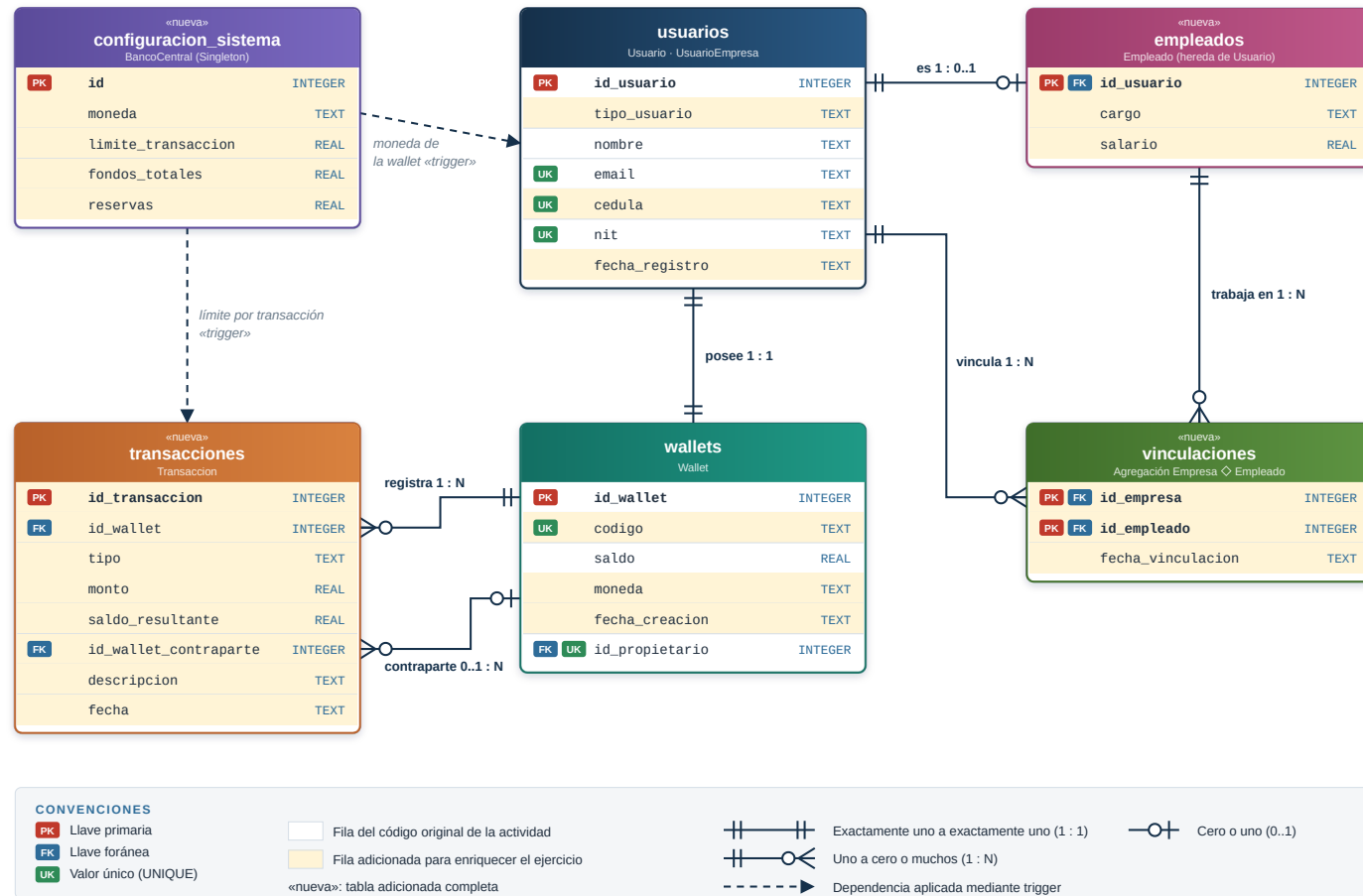


Figura 2. Modelo entidad-relación de la Quantum Wallet en notación de pata de gallo. Fondo blanco: código original de la actividad; fondo amarillo: elementos adicionados para enriquecer el ejercicio. Elaborado en formato .drawio con Draw.io Integration para Visual Studio Code.

Estructura de cada tabla

Las tablas siguientes detallan los campos, su tipo y sus restricciones. Las filas con fondo amarillo corresponden a los campos adicionados para enriquecer el ejercicio.

usuarios

CAMPO	TIPO	RESTRICCIONES	PROPÓSITO
id_usuario	INTEGER	PRIMARY KEY AUTOINCREMENT	Identificador único; no se reutiliza al eliminar un registro.
tipo_usuario	TEXT	NOT NULL; CHECK: PERSONA, EMPRESA o EMPLEADO	Indica la clase de origen del registro.
nombre	TEXT	NOT NULL; CHECK: no vacío	Nombre de la persona o razón social de la empresa.
email	TEXT	UNIQUE; CHECK: formato de correo	Correo de contacto, opcional como en el constructor de la clase.
cedula	TEXT	UNIQUE; obligatoria para empleados	Documento de identidad de personas y empleados.
nit	TEXT	UNIQUE; obligatorio solo para empresas	Identificación tributaria de las empresas.
fecha_registro	TEXT	NOT NULL; DEFAULT fecha actual	Trazabilidad del alta del usuario.

wallets

CAMPO	TIPO	RESTRICCIONES	PROPÓSITO
id_wallet	INTEGER	PRIMARY KEY AUTOINCREMENT	Identificador único de la billetera.
codigo	TEXT	NOT NULL; UNIQUE	Código visible de la billetera (W-00001).
saldo	REAL	NOT NULL; DEFAULT 0; CHECK saldo >= 0	Dinero disponible, con centavos. La restricción impide saldos negativos.
moneda	TEXT	NOT NULL; DEFAULT 'COP'; CHECK: 3 caracteres	Código de moneda según la norma ISO 4217.
fecha_creacion	TEXT	NOT NULL; DEFAULT fecha actual	Fecha de apertura de la billetera.
id_propietario	INTEGER	NOT NULL; UNIQUE; FK → usuarios ON DELETE CASCADE	Dueño de la billetera. UNIQUE garantiza la relación uno a uno.

transacciones

CAMPO	TIPO	RESTRICCIONES	PROPÓSITO
id_transaccion	INTEGER	PRIMARY KEY AUTOINCREMENT	Número consecutivo del movimiento.
id_wallet	INTEGER	NOT NULL; FK → wallets ON DELETE CASCADE	Billetera a la que pertenece el movimiento.

tipo	TEXT	CHECK: RECARGA, PAGO, TRANSFERENCIA_ENVIADA o TRANSFERENCIA_RECIBIDA	Naturaleza del movimiento; determina si suma o resta.
monto	REAL	NOT NULL; CHECK monto > 0	Valor del movimiento, siempre positivo.
saldo_resultante	REAL	NOT NULL; CHECK >= 0	Saldo de la billetera después del movimiento, como en la clase <code>Transaccion</code> .
id_wallet_contraparte	INTEGER	FK → wallets ON DELETE SET NULL; distinta de id_wallet	Billetera de origen o de destino en las transferencias.
descripcion	TEXT	—	Concepto del movimiento.
fecha	TEXT	NOT NULL; DEFAULT fecha actual	Fecha y hora del movimiento.

empleados

CAMPO	TIPO	RESTRICCIONES	PROPÓSITO
id_usuario	INTEGER	PRIMARY KEY; FK → usuarios ON DELETE CASCADE	El empleado es un usuario: comparte su identificador.
cargo	TEXT	NOT NULL	Cargo dentro de la empresa.
salario	REAL	NOT NULL; CHECK salario > 0	Valor que se transfiere en el pago de la nómina.

vinculaciones

CAMPO	TIPO	RESTRICCIONES	PROPÓSITO
id_empresa	INTEGER	PRIMARY KEY compuesta; FK → usuarios ON DELETE CASCADE	Empresa que vincula al empleado.
id_empleado	INTEGER	PRIMARY KEY compuesta; FK → empleados ON DELETE CASCADE	Empleado vinculado. La llave compuesta impide registrar dos veces el mismo vínculo.
fecha_vinculacion	TEXT	NOT NULL; DEFAULT fecha actual	Fecha de inicio del vínculo.

configuracion_sistema

CAMPO	TIPO	RESTRICCIONES	PROPÓSITO
id	INTEGER	PRIMARY KEY; CHECK id = 1	Solo admite una fila: la configuración es única.
moneda	TEXT	NOT NULL; DEFAULT 'COP'	Moneda oficial que reciben las nuevas billeteras.
limite_transaccion	REAL	NOT NULL; CHECK > 0	Monto máximo permitido por movimiento (50.000.000).
fondos_totales	REAL	NOT NULL; DEFAULT 0	Fondos del Banco Central.
reservas	REAL	NOT NULL; DEFAULT 0	Reservas emitidas por el Banco Central.

Relaciones, integridad y normalización

Las relaciones entre tablas se implementaron con llaves foráneas. El comportamiento al eliminar un registro principal reproduce el tipo de relación definido en el modelo de clases.

RELACIÓN	CARDINALIDAD	IMPLEMENTACIÓN	AL ELIMINAR EL REGISTRO PRINCIPAL
usuarios – wallets	1 : 1	FK <code>id_propietario</code> con <code>NOT NULL</code> y <code>UNIQUE</code>	Se elimina la billetera (<code>CASCADE</code>). Corresponde a la composición: la billetera no existe sin su dueño.
wallets – transacciones	1 : N	FK <code>id_wallet</code>	Se elimina el historial (<code>CASCADE</code>), también por composición.
wallets – transacciones (contraparte)	0..1 : N	FK <code>id_wallet_contraparte</code>	El movimiento se conserva y la referencia queda nula (<code>SET NULL</code>), para no alterar el historial de la otra billetera.
usuarios – empleados	1 : 0..1	Llave primaria compartida	Se elimina la ficha del empleado (<code>CASCADE</code>).
empresas – empleados	N : M	Tabla <code>vinculaciones</code> con llave compuesta	Se elimina el vínculo, pero el empleado se conserva. Corresponde a la agregación: la parte existe por fuera del todo.

ACTIVACIÓN DE LAS LLAVES FORÁNEAS

SQLite no verifica las llaves foráneas por defecto. Por esa razón, `configurar_db.py` ejecuta `PRAGMA foreign_keys = ON;` en cada conexión; sin esa instrucción, las restricciones de la tabla serían solo declarativas.

Reglas aplicadas mediante triggers

Algunas reglas del modelo de clases involucran más de una tabla y no pueden expresarse con una restricción de columna. Para ellas se definieron triggers, que la base de datos ejecuta automáticamente antes o después de una inserción.

TRIGGER	MOMENTO	REGLA
<code>trg_usuario_crea_wallet</code>	Después de insertar un usuario	Crea su billetera con el código y la moneda del sistema, del mismo modo que el constructor de <code>Usuario</code> crea su <code>Wallet</code> .
<code>trg_transaccion_valida_limite</code>	Antes de insertar una transacción	Rechaza montos superiores al límite configurado; equivale a <code>BancoCentral.validar_monto()</code> .
<code>trg_empleado_valida_tipo</code>	Antes de insertar un empleado	Exige que el usuario sea de tipo EMPLEADO.
<code>trg_vinculacion_valida_empresa</code>	Antes de insertar un vínculo	Exige que quien vincula sea una EMPRESA.

Vistas e índices

Se definieron tres vistas, que almacenan las consultas de uso más frecuente, y dos índices sobre las columnas que se consultan con mayor regularidad.

OBJETO	CONTENIDO O PROPÓSITO
<code>vista_saldos</code>	Cada usuario con su billetera y su saldo.
<code>vista_nomina</code>	Empresas con sus empleados, cargos y salarios.
<code>vista_movimientos</code>	Historial de transacciones con el nombre del titular.
<code>idx_transacciones_wallet_fecha</code>	Consulta del historial de una billetera en orden cronológico.
<code>idx_vinculaciones_Empleado</code>	Búsqueda de las empresas de un empleado.

Normalización

Primera forma normal. Todos los campos contienen valores atómicos. Las listas `historial` y `empleados` se trasladaron a tablas propias en lugar de almacenarse en una sola columna.

Segunda forma normal. La única llave compuesta del esquema está en `vinculaciones`, y su único campo adicional, `fecha_vinculacion`, depende de la llave completa: describe el vínculo, no la empresa ni el empleado por separado.

Tercera forma normal. Los datos del propietario no se repiten en `wallets` ni en `transacciones`; se obtienen mediante `JOIN` a partir de las llaves foráneas.

DESNORMALIZACIÓN CONTROLADA DEL SALDO

El saldo podría calcularse a partir del historial de transacciones. Se mantiene como columna porque es el atributo principal de `wallet` y la consulta más frecuente del sistema. Su consistencia se garantiza porque cada movimiento actualiza el saldo y registra la transacción dentro de una misma transacción de base de datos; además, el script concilia ambos valores al finalizar.

El script configurar_db.py

El script crea la base de datos desde cero, inserta los datos de prueba y ejecuta las verificaciones. Su estructura se organiza en los bloques siguientes.

BLOQUE	FUNCIÓN
ESQUEMA_SQL	Sentencias SQL de creación de tablas, índices, triggers y vistas.
conectar() , crear_esquema()	Abren la conexión con las llaves foráneas activas y recrean el archivo en cada ejecución.
registrar_usuario() , registrar_empleado() , vincular_empleado()	Registran usuarios, empleados y vínculos con parámetros seguros (?).
recargar() , realizar_pago() , transferir() , pagar_nomina()	Equivalen a los métodos de las clases; cada una actualiza el saldo y registra el movimiento de forma atómica.
insertar_datos_prueba()	Escenarios del código original (Ana Torres, Bancolombia) y de la versión ampliada (Quantum SAS).
verificar() , conciliar() , probar_integridad() , probar_autoincrement_y_cascada()	Comprueban la estructura, la conciliación de saldos, las restricciones y AUTOINCREMENT .

Definición de la tabla wallets y del trigger de composición

```
CREATE TABLE wallets (
  id_wallet      INTEGER PRIMARY KEY AUTOINCREMENT,
  codigo         TEXT NOT NULL UNIQUE,           -- [AGREGADO] id_billetera
  saldo          REAL NOT NULL DEFAULT 0
  CONSTRAINT ck_wallet_saldo_no_negativo CHECK (saldo >= 0),
  moneda         TEXT NOT NULL DEFAULT 'COP'     -- [AGREGADO]
  CONSTRAINT ck_wallet_moneda CHECK (length(moneda) = 3),
  fecha_creacion TEXT NOT NULL DEFAULT (datetime('now', 'localtime')), -- [AGREGADO]
  id_propietario INTEGER NOT NULL UNIQUE,
  FOREIGN KEY (id_propietario) REFERENCES usuarios(id_usuario)
  ON DELETE CASCADE ON UPDATE CASCADE
);

CREATE TRIGGER trg_usuario_crea_wallet
AFTER INSERT ON usuarios
BEGIN
  INSERT INTO wallets (codigo, moneda, id_propietario)
  VALUES ('W-' || printf('%05d', NEW.id_usuario),
    (SELECT moneda FROM configuracion_sistema WHERE id = 1),
    NEW.id_usuario);
END;
```

Operación atómica: transferencia entre billeteras

```
def transferir(conexion, id_origen, id_destino, monto, descripcion="Transferencia",
              fecha=None): # [AGREGADO]
    """Debito y credito en una sola transaccion: o pasan los dos o ninguno."""
    with conexion:
        w_origen = _id_wallet(conexion, id_origen)
        w_destino = _id_wallet(conexion, id_destino)
        _mover_saldo(conexion, w_origen, monto, "TRANSFERENCIA_ENVIADA",
                    descripcion, w_destino, fecha)
        _mover_saldo(conexion, w_destino, monto, "TRANSFERENCIA_RECIBIDA",
                    descripcion, w_origen, fecha)
```

ATOMICIDAD DE LAS OPERACIONES

El bloque `with conexion` confirma los cambios solo si todas las sentencias se ejecutan sin error; en caso contrario, los revierte. En una transferencia sin saldo suficiente, la restricción `CHECK (saldo >= 0)` rechaza el débito y la base de datos descarta también el crédito, por lo que el dinero no aparece ni desaparece. La función auxiliar `_mover_saldo()` recibe el monto siempre positivo y determina si suma o resta según el tipo de movimiento; de este modo, un monto negativo es rechazado por la restricción `ck_transaccion_monto_positivo` de la base de datos.

Creación de la base de datos

La ejecución de `python3 configurar_db.py` en la carpeta del proyecto genera el archivo `quantum_wallet.db` e imprime el resultado de cada etapa.

Figura 3. Ejecución de `configurar_db.py`: versión de SQLite, objetos creados, saldos por usuario, nómina y conciliación de saldos.

```
[4] Pruebas de integridad (todas deben ser RECHAZADAS)
RECHAZADA Wallet con un propietario que no existe (FK)
-> IntegrityError: FOREIGN KEY constraint failed
RECHAZADA Segunda wallet para el mismo usuario (relacion 1 a 1)
-> IntegrityError: UNIQUE constraint failed: wallets.id_propietario
RECHAZADA Correo electrónico repetido (UNIQUE)
-> IntegrityError: UNIQUE constraint failed: usuarios.email
RECHAZADA Empresa registrada sin NIT (CHECK)
-> IntegrityError: CHECK constraint failed: ck_usuario_nit_empresa
RECHAZADA Pago mayor al saldo disponible (CHECK saldo >= 0)
-> IntegrityError: CHECK constraint failed: ck_wallet_saldo_no_negativo
RECHAZADA Recarga que supera el límite del Banco Central (trigger)
-> IntegrityError: el monto supera el límite por transaccion
RECHAZADA Segunda fila de configuración (Singleton)
-> IntegrityError: CHECK constraint failed: ck_config_fila_unica
RECHAZADA Ficha de empleado para un usuario PERSONA (trigger)
-> IntegrityError: el usuario no es de tipo EMPLEADO
Resultado: 8 de 8 operaciones inválidas rechazadas

[5] AUTOINCREMENT y borrado en cascada
Se creo y se borro el usuario 6; wallets huérfanas: 0
El siguiente usuario recibió el id 7 (el 6 no se reutiliza)
```

Figura 4. Continuación de la ejecución: pruebas de integridad y comprobación de AUTOINCREMENT y del borrado en cascada.

INTERPRETACIÓN

El script creó las seis tablas, los dos índices, los cuatro triggers y las tres vistas. Los cinco usuarios recibieron su billetera de forma automática y los saldos coinciden con el historial de once transacciones. La billetera de Ana Torres registra 220.000,50 y la de Laura Gomez 4.349.999,50, lo que confirma que el tipo `REAL` conserva los centavos de la transferencia de 150.000,50. Las ocho operaciones inválidas fueron rechazadas.

Inspección con DB Browser for SQLite

La base de datos generada se abrió con DB Browser for SQLite para verificar su estructura y su contenido de forma independiente del script.

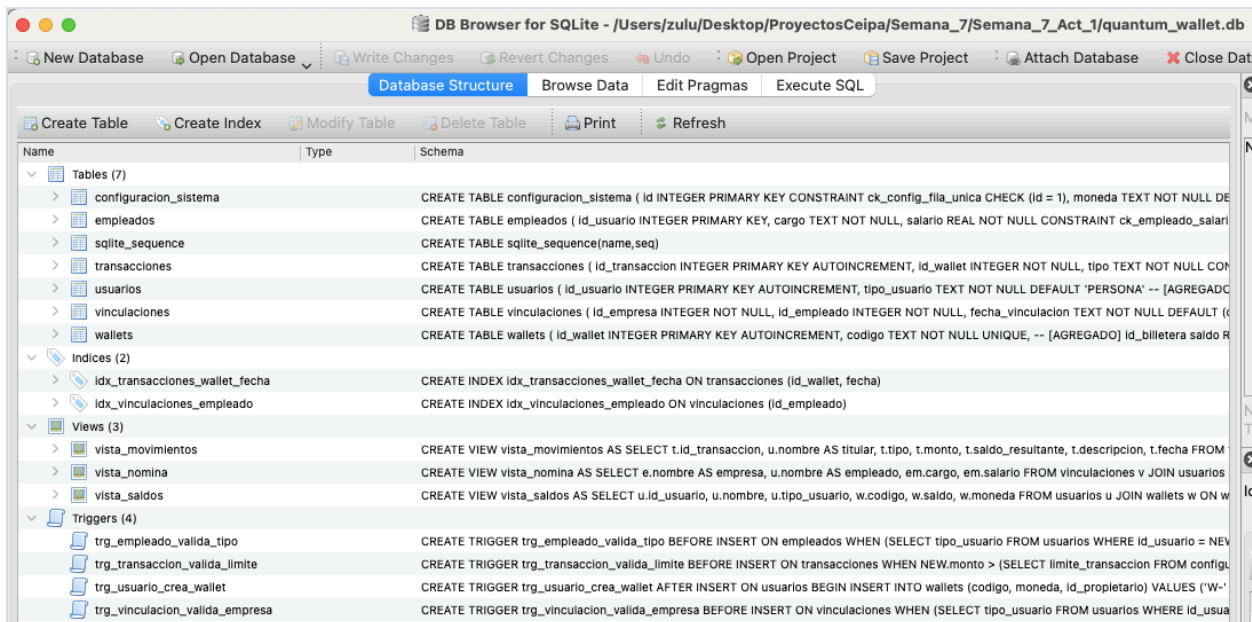


Figura 5. Pestaña Database Structure: tablas, índices, vistas y triggers de quantum_wallet.db, con la ruta del archivo en la barra de título.

La herramienta lista siete tablas porque incluye `sqlite_sequence`, una tabla interna que SQLite crea al usar `AUTOINCREMENT` para registrar el último identificador asignado a cada tabla.

Database Structure Browse Data Edit Pragmas E							
Table: usuarios							
	id_usuario	tipo_usuario	nombre	email	cedula	nit	fecha_registro
	Filter	Filter	Filter	Filter	Filter	Filter	Filter
1	1	PERSONA	Ana Torres	ana@quantum.com	NULL	NULL	2026-10-02 20:51:55
2	2	EMPRESA	Bancolombia	contacto@bancolombia.com	NULL	900123456-7	2026-10-02 20:51:55
3	3	EMPRESA	Quantum SAS	pagos@quantum.com	NULL	901555222-1	2026-10-02 20:51:55
4	4	EMPLEADO	Laura Gomez	laura.gomez@quantum.com	1036123456	NULL	2026-10-02 20:51:55
5	5	EMPLEADO	Pedro Rios	NULL	1017987654	NULL	2026-10-02 20:51:55

Figura 6. Tabla usuarios: tres tipos de usuario, NIT solo en las empresas y cédula en los empleados.

Database Structure Browse Data						
Table: wallets						
<u>id_wallet</u>	codigo	saldo	moneda	fecha_creacion	<i>id_propietario</i>	
Filter	Filter	Filter	Filter	Filter	Filter	
1	1	W-00001	220000.5	COP	2026-10-02 20:51:55	1
2	2	W-00002	300000.0	COP	2026-10-02 20:51:55	2
3	3	W-00003	2300000.0	COP	2026-10-02 20:51:55	3
4	4	W-00004	4349999.5	COP	2026-10-02 20:51:55	4
5	5	W-00005	3200000.0	COP	2026-10-02 20:51:55	5

Figura 7. Tabla wallets: una billetera por usuario, vinculada por la llave foránea *id_propietario* (en cursiva).

Database Structure Browse Data Edit Pragmas Execute SQL								
Table: transacciones								
<u>id_transaccion</u>	<i>id_wallet</i>	tipo	monto	saldo_resultante	<i>id_wallet_contraparte</i>	descripcion	fecha	
Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter	
1	1	1	RECARGA	100000.0	100000.0	NULL	Recarga	2026-10-02 20:51:55
2	2	1	PAGO	30000.0	70000.0	NULL	Pago en comercio	2026-10-02 20:51:55
3	3	2	RECARGA	500000.0	500000.0	NULL	Recarga	2026-10-02 20:51:55
4	4	2	PAGO	200000.0	300000.0	NULL	Pago a proveedor	2026-10-02 20:51:55
5	5	3	RECARGA	10000000.0	10000000.0	NULL	Capital de trabajo	2026-10-02 20:51:55
6	6	3	TRANSFERENCIA_ENVIADA	4500000.0	5500000.0	4	Pago de nomina	2026-10-02 20:51:55
7	7	4	TRANSFERENCIA_RECIBIDA	4500000.0	4500000.0	3	Pago de nomina	2026-10-02 20:51:55
8	8	3	TRANSFERENCIA_ENVIADA	3200000.0	2300000.0	5	Pago de nomina	2026-10-02 20:51:55
9	9	5	TRANSFERENCIA_RECIBIDA	3200000.0	3200000.0	3	Pago de nomina	2026-10-02 20:51:55
10	10	4	TRANSFERENCIA_ENVIADA	150000.5	4349999.5	1	Devolucion de prestamo	2026-10-02 20:51:55
11	11	1	TRANSFERENCIA_RECIBIDA	150000.5	220000.5	4	Devolucion de prestamo	2026-10-02 20:51:55

Figura 8. Tabla transacciones: recargas, pagos, nómina y transferencia con centavos; cada movimiento registra el saldo resultante y la billetera contraparte.

RESULTADO DE LA VERIFICACIÓN

DB Browser for SQLite identifica la llave primaria de cada tabla (subrayada) y las llaves foráneas (en cursiva). Los datos coinciden con la salida del script, y los valores nulos aparecen únicamente donde el modelo los admite: el NIT de las personas, la cédula de las empresas y la contraparte de las recargas y los pagos.

Operaciones inválidas rechazadas

Para comprobar que el esquema protege la información, el script intenta ocho operaciones que violan las reglas del modelo. En todos los casos la base de datos las rechazó con un error de integridad y revirtió los cambios.

OPERACIÓN INTENTADA	MECANISMO	MENSAJE DE SQLITE
Billetera con un propietario inexistente	Llave foránea	FOREIGN KEY constraint failed
Segunda billetera para el mismo usuario	UNIQUE en id_propietario	UNIQUE constraint failed: wallets.id_propietario
Correo electrónico repetido	UNIQUE en email	UNIQUE constraint failed: usuarios.email
Empresa registrada sin NIT	CHECK	CHECK constraint failed: ck_usuario_nit_empresa
Pago mayor al saldo disponible	CHECK	CHECK constraint failed: ck_wallet_saldo_no_negativo
Recarga superior al límite del Banco Central	Trigger	el monto supera el limite por transaccion
Segunda fila de configuración	CHECK (id = 1)	CHECK constraint failed: ck_config_fila_unica
Ficha de empleado para un usuario PERSONA	Trigger	el usuario no es de tipo EMPLEADO

AUTOINCREMENT y borrado en cascada

El script registró un usuario temporal con identificador 6, le hizo una recarga y luego lo eliminó. La consulta posterior no encontró billeteras huérfanas, lo que confirma que `ON DELETE CASCADE` eliminó la billetera y su historial. El siguiente usuario recibió el identificador 7: `AUTOINCREMENT` no reutiliza los identificadores eliminados, lo que evita que un registro nuevo herede referencias de uno anterior. Ambos usuarios temporales se retiraron al finalizar, de modo que los datos de prueba quedan limpios.

Carga masiva de registros desde Excel

Como mejora del ejercicio, se incorporó un proceso de carga masiva que importa a `quantum_wallet.db` un archivo de Excel con 560 registros simulados. El proceso responde a una necesidad habitual de una Fintech: incorporar en bloque información proveniente de otros sistemas sin perder el control sobre la calidad de los datos.

COMPONENTE	FUNCIÓN
<code>generar_excel.py</code>	Genera el archivo de carga; su semilla fija produce el mismo archivo en cada ejecución.
<code>carga_masiva_quantum.xlsx</code>	Hoja <code>usuarios</code> (60 filas) y hoja <code>movimientos</code> (500 filas).
<code>importar_excel.py</code>	Lee el archivo, valida cada fila y la importa con las funciones de <code>configurar_db.py</code> .
<code>errores_importacion.log</code>	Registro de cada carga y de cada rechazo, con su hoja, fila y motivo.
<code>resultado_importacion.xlsx</code>	Filas rechazadas, para su revisión y corrección por el área operativa.
Tabla <code>importaciones</code>	Control de cargas; la huella SHA-256 impide importar dos veces el mismo archivo.

Datos simulados

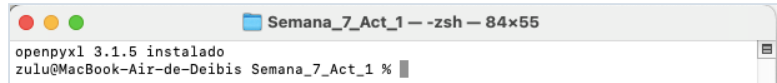
La hoja `usuarios` contiene 33 personas, 9 empresas y 14 empleados vinculados a una empresa, con nombres, cédulas y NIT con el formato colombiano. La hoja `movimientos` recorre septiembre de 2026: una recarga inicial por usuario, pagos en comercios y a proveedores, transferencias entre usuarios y el pago de la nómina el día 30. El generador lleva el saldo de cada usuario durante la simulación, de modo que ningún movimiento válido supera el saldo disponible.

Para comprobar los controles, 29 filas (el 5 % de la carga) contienen errores intencionales:

ERROR INCLUIDO EN EL ARCHIVO	FILAS	MECANISMO QUE LO DETECTA
Pago mayor al saldo disponible	6	CHECK <code>ck_wallet_saldo_no_negativo</code>
Documento de destino no registrado	5	Validación del importador
Monto negativo	4	CHECK <code>ck_transaccion_monto_positivo</code>
Recarga superior al límite del Banco Central	3	Trigger <code>trg_transaccion_valida_limite</code>
Transferencia a la misma billetera	3	CHECK <code>ck_transaccion_contraparte</code>
Fecha inexistente (31 de septiembre)	2	Validación del importador
Tipo de movimiento no válido (RETIRO)	2	Validación del importador
Empresa sin NIT	1	CHECK <code>ck_usuario_nit_empresa</code>
Correo electrónico duplicado	1	UNIQUE en <code>usuarios.email</code>
Empleado sin cédula	1	CHECK <code>ck_usuario_cedula_empleado</code>
Empleado de una empresa no registrada	1	Validación del importador

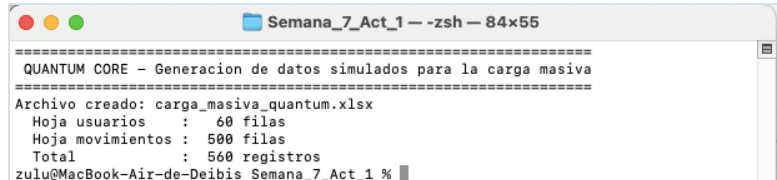
Preparación del entorno y generación del archivo

La lectura de archivos `.xlsx` requiere la librería `openpyxl`, que se instaló con `python3 -m pip install openpyxl`. Esta forma de instalación garantiza que la librería quede disponible para el mismo intérprete que ejecuta los scripts.



```
openpyxl 3.1.5 instalado
zulu@MacBook-Air-de-Deibis Semana_7_Act_1 %
```

Figura 9. Verificación de la librería openpyxl 3.1.5 en la Terminal.



```
=====
QUANTUM CORE - Generacion de datos simulados para la carga masiva
=====
Archivo creado: carga_masiva_quantum.xlsx
Hoja usuarios : 60 filas
Hoja movimientos : 500 filas
Total : 560 registros
zulu@MacBook-Air-de-Deibis Semana_7_Act_1 %
```

Figura 10. Ejecución de generar_excel.py: 60 usuarios y 500 movimientos, 560 registros en total.

A1	tipo_usuario							
	A	B	C	D	E	F	G	H
1	tipo_usuario	nombre	email	cedula	nit	cargo	salario	nit_empresa
2	PERSONA	Paula Valencia Lopera	paula.valencia46@correo.com	1020771657				
3	PERSONA	Natalia Restrepo Velez	natalia.restrepo47@correo.com	1061884106				
4	PERSONA	Diego Rios Lopera	diego.rios10@correo.com	1097054446				
5	PERSONA	Ana Maria Henao Alzate	ana.henao29@correo.com	1063482774				
6	PERSONA	Sara Ramirez Mejia	sara.ramirez75@correo.com	1074416889				
7	EMPLEADO	Ana Maria Botero Montoya		1043726695		Coordinador	5.800.000,00	996865468-1
8	EMPRESA	Consultores Llanogrande SAS	info@consultores.co					
9	EMPLEADO	Sofia Rios Henao		1052341080		Analista	3.200.000,00	996865468-1
10	PERSONA	Carlos Alzate Mejia	carlos.alzate86@correo.com	1041289034				
11	PERSONA	David Jaramillo Zuluaga	david.jaramillo57@correo.com	1092520726				
12	EMPRESA	Logistica Andina SAS	pagos@logisticaandina.co		996865468-1			
13	PERSONA	Luisa Rios Cardona	luisa.rios47@correo.com	1026821744				
14	EMPLEADO	Empleado Sin Empresa		1012945288		Analista	3.200.000,00	999999999-9
15	EMPLEADO	Jorge Jaramillo Zuluaga		1036753198		Operario	1.550.000,00	942203407-0
16	PERSONA	Jorge Rios Rios	jorge.rios4@correo.com	1077244482				
17	PERSONA	Mariana Zuluaga Montoya	mariana.zuluaga13@correo.com	1060351650				
18	EMPLEADO	Daniela Cardona Jaramillo		1020654844		Operario	1.550.000,00	996865468-1
19	PERSONA	Alejandro Salazar Rios	alejandro.salazar38@correo.com	1011332433				
20	PERSONA	Felipe Henao Gomez	felipe.henao91@correo.com	1070224935				
21	PERSONA	Carlos Cardona Jaramillo	carlos.cardona66@correo.com	1066727300				
22	EMPLEADO	Nicolas Mejia Restrepo		1009856055		Auxiliar administrativo	1.750.000,00	983526008-7
23	EMPLEADO	Valentina Velez Lopera		1003433596		Desarrollador	4.500.000,00	925988000-5
24	PERSONA	Registro Duplicado	mariana.zuluaga13@correo.com	1007692234				
25	PERSONA	Luisa Valencia Cardona	luisa.valencia92@correo.com	1073233523				
26	EMPLEADO	Miguel Lopera Giraldo		1017332847		Coordinador	5.800.000,00	925988000-5
27	EMPLEADO	Empleado Sin Cedula				Operario	1.550.000,00	925988000-5
28	PERSONA	Jorge Ramirez Lopera	jorge.ramirez35@correo.com	1066896901				
29	PERSONA	Jorge Mejia Botero	jorge.mejia26@correo.com	1039510228				
30	EMPLEADO	Nicolas Lopera Ospina		1006467611		Auxiliar administrativo	1.750.000,00	942203407-0
31	EMPRESA	TecnoRetiro SAS	pagos@tecnoretiro.co		939968745-9			
32	EMPRESA	Lacteos El Carmen SAS	pagos@lacteoselcarmen.co		993414184-8			
33	PERSONA	Catalina Zuluaga Giraldo	catalina.zuluaga96@correo.com	1055205206				
34	EMPLEADO	Sofia Botero Salazar		1080291676		Desarrollador	4.500.000,00	988721495-7
35	EMPRESA	Panaderia San Antonio SAS	pagos@panaderiasanantonio.co		942203407-0			
36	PERSONA	Carlos Jaramillo Ospina	carlos.jaramillo55@correo.com	1097707066				
37	PERSONA	Ana Maria Mejia Rios	ana.mejia34@correo.com	1056366902				
38	PERSONA	Andres Castano Valencia	andres.castano75@correo.com	1077079038				
usuarios movimientos +								

Figura 11. Hoja usuarios del archivo de carga. Se observan registros con errores intencionales, como «Empleado Sin Empresa» (fila 14), «Registro Duplicado» (fila 24) y «Empleado Sin Cedula» (fila 27).

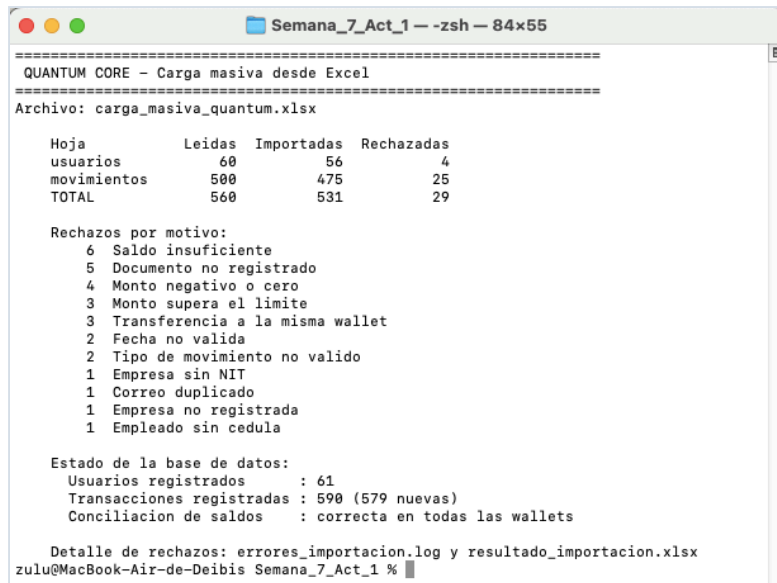
A1						fecha
	A	B	C	D	E	F
1	fecha	tipo	documento_origen	documento_destino	monto	descripcion
2	2026-09-01 08:00	RECARGA	925988000-5		28.000.000,00	Recarga inicial
3	2026-09-01 08:07	RECARGA	977446522-8		42.500.000,00	Recarga inicial
4	2026-09-01 08:14	RECARGA	996865468-1		43.000.000,00	Recarga inicial
5	2026-09-01 08:21	RECARGA	939968745-9		36.000.000,00	Recarga inicial
6	2026-09-01 08:28	RECARGA	993414184-8		35.000.000,00	Recarga inicial
7	2026-09-01 08:35	RECARGA	966453983-9		39.500.000,00	Recarga inicial
8	2026-09-01 08:42	RECARGA	983526008-7		44.000.000,00	Recarga inicial
9	2026-09-01 08:49	RECARGA	988721495-7		44.500.000,00	Recarga inicial
10	2026-09-01 08:56	RECARGA	942203407-0		26.000.000,00	Recarga inicial
11	2026-09-01 09:03	RECARGA	1060351650		3.350.000,00	Recarga inicial
12	2026-09-01 09:10	RECARGA	1042175656		2.800.000,00	Recarga inicial
13	2026-09-01 09:17	RECARGA	1047866880		4.650.000,00	Recarga inicial
14	2026-09-01 09:24	RECARGA	1097054446		2.150.000,00	Recarga inicial
15	2026-09-01 09:31	RECARGA	1039207934		4.600.000,00	Recarga inicial
16	2026-09-01 09:38	RECARGA	1094485997		3.750.000,00	Recarga inicial
17	2026-09-01 09:45	RECARGA	1061884106		3.500.000,00	Recarga inicial
18	2026-09-01 09:52	RECARGA	1074416889		1.550.000,00	Recarga inicial
19	2026-09-01 09:59	RECARGA	1080552815		3.150.000,00	Recarga inicial
20	2026-09-01 10:06	RECARGA	1056366902		4.000.000,00	Recarga inicial
21	2026-09-01 10:13	RECARGA	1077244482		2.400.000,00	Recarga inicial
22	2026-09-01 10:20	RECARGA	1070224935		3.250.000,00	Recarga inicial
23	2026-09-01 10:27	RECARGA	1019114289		2.650.000,00	Recarga inicial
24	2026-09-01 10:34	RECARGA	1094047344		3.050.000,00	Recarga inicial
25	2026-09-01 10:41	RECARGA	1073233523		1.150.000,00	Recarga inicial
26	2026-09-01 10:48	RECARGA	1041289034		3.550.000,00	Recarga inicial
27	2026-09-01 10:55	RECARGA	1039510228		950.000,00	Recarga inicial
28	2026-09-01 11:02	RECARGA	1087998874		2.600.000,00	Recarga inicial
29	2026-09-01 11:09	RECARGA	1026821744		2.900.000,00	Recarga inicial
30	2026-09-01 11:16	RECARGA	1066727300		2.650.000,00	Recarga inicial
31	2026-09-01 11:23	RECARGA	1038932423		1.550.000,00	Recarga inicial
32	2026-09-01 11:30	RECARGA	1092520726		2.700.000,00	Recarga inicial
33	2026-09-01 11:37	RECARGA	1097707066		3.450.000,00	Recarga inicial
34	2026-09-01 11:44	RECARGA	1066896901		2.800.000,00	Recarga inicial
35	2026-09-01 11:51	RECARGA	1055205206		4.500.000,00	Recarga inicial
36	2026-09-01 11:58	RECARGA	1000028718		900.000,00	Recarga inicial
37	2026-09-01 12:05	RECARGA	1081890761		4.250.000,00	Recarga inicial
38	2026-09-01 12:12	RECARGA	1005994039		2.950.000,00	Recarga inicial
	usuarios	movimientos				

Figura 12. Hoja movimientos del archivo de carga, iniciada con las recargas del 1 de septiembre.

Diseño del importador

DECISIÓN	JUSTIFICACIÓN
Reutilizar las funciones del sistema	El importador no inserta directamente en las tablas: invoca <code>registrar_usuario()</code> , <code>recargar()</code> , <code>realizar_pago()</code> y <code>transferir()</code> . Así, cada fila del Excel pasa por las mismas llaves, restricciones y triggers que una operación individual; por ejemplo, cada usuario importado recibe su billetera mediante el trigger.
Atomicidad por fila	Cada fila se importa completa o no se importa. Una transferencia rechazada no deja un débito sin su crédito, y un empleado rechazado no deja un usuario sin su vínculo.
Orden de dependencias	Los empleados se cargan después de personas y empresas, porque su vínculo exige que la empresa exista. Dentro de cada grupo se respeta el orden del archivo.
Dos niveles de validación	El importador valida el formato (fecha, tipo de movimiento y existencia del documento). Las reglas de negocio (saldo, montos, límite, unicidad) las aplica la base de datos.
Trazabilidad	El módulo <code>logging</code> registra cada carga en modo de agregar, de modo que el log conserva el historial. Los rechazos se exportan además a Excel para su corrección.
Control de duplicados	Antes de importar se calcula la huella SHA-256 del archivo. Si ya existe en la tabla <code>importaciones</code> , la carga se rechaza completa y se evita duplicar los movimientos.
Fechas originales	Las operaciones recibieron un parámetro <code>fecha</code> , de modo que cada movimiento conserva la fecha registrada en el archivo y no la del momento de la importación.

Resultados de la importación



```
=====
QUANTUM CORE - Carga masiva desde Excel
=====
Archivo: carga_masiva_quantum.xlsx

Hoja      Leidas  Importadas  Rechazadas
usuarios    60      56          4
movimientos 500     475        25
TOTAL      560     531        29

Rechazos por motivo:
6 Saldo insuficiente
5 Documento no registrado
4 Monto negativo o cero
3 Monto supera el limite
3 Transferencia a la misma wallet
2 Fecha no valida
2 Tipo de movimiento no valido
1 Empresa sin NIT
1 Correo duplicado
1 Empresa no registrada
1 Empleado sin cedula

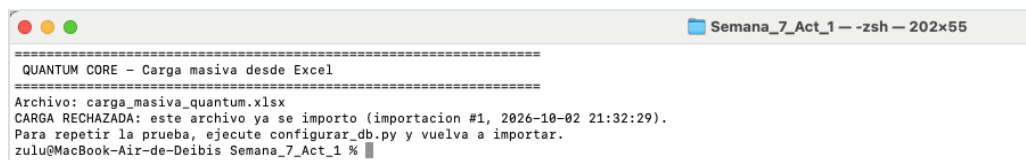
Estado de la base de datos:
Usuarios registrados      : 61
Transacciones registradas : 590 (579 nuevas)
Conciliacion de saldos    : correcta en todas las wallets

Detalle de rechazos: errores_importacion.log y resultado_importacion.xlsx
zulu@MacBook-Air-de-Deibis Semana_7_Act_1 %
```

Figura 13. Ejecución de `importar_excel.py`: 560 filas leídas, 531 importadas y 29 rechazadas, con el detalle por motivo y la conciliación de saldos.

INTERPRETACIÓN

El importador rechazó exactamente las 29 filas con errores intencionales y aceptó las 531 válidas. Los 475 movimientos importados generaron 579 registros en la tabla `transacciones`, porque cada una de las 104 transferencias registra dos movimientos: el débito en la billetera de origen y el crédito en la de destino. La conciliación final confirma que el saldo de cada billetera coincide con su historial.



```
=====
QUANTUM CORE - Carga masiva desde Excel
=====
Archivo: carga_masiva_quantum.xlsx
CARGA RECHAZADA: este archivo ya se importo (importacion #1, 2026-10-02 21:32:29).
Para repetir la prueba, ejecute configurar_db.py y vuelva a importar.
zulu@MacBook-Air-de-Deibis Semana_7_Act_1 %
```

Figura 14. Segundo intento de importar el mismo archivo: la carga se rechaza porque su huella SHA-256 ya está registrada.

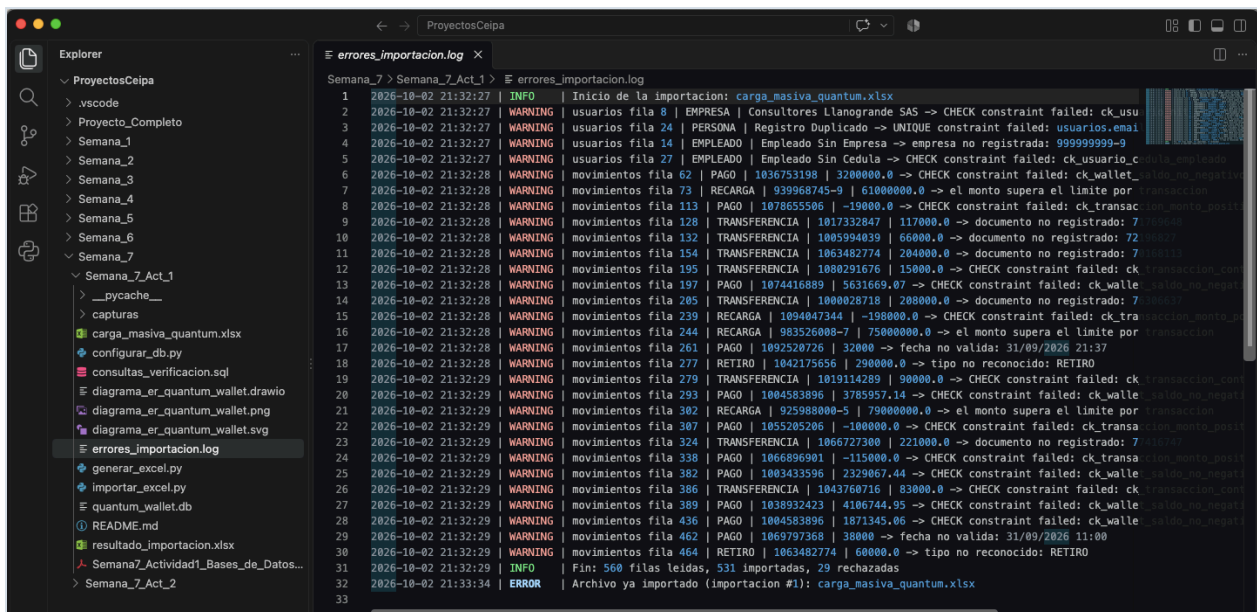


Figura 15. Archivo errores_importacion.log en Visual Studio Code: inicio de la carga, una línea por rechazo con la hoja, la fila y el motivo, resumen final y registro del intento repetido.

Autoguardado

resultado_importacion

Sin etiqueta

Inicio

Insertar

Dibujar

Disposición de página

Fórmulas

Datos

Revisar

Vista

Automatizar

Pegar

Calibri

11

A⁺

A⁻

N

K

S

U

</

Figura 16. Archivo resultado_importacion.xlsx con las 29 filas rechazadas, su motivo y el detalle técnico.

	id_transaccion	id_wallet	tipo	monto	saldo_resultante	id_wallet_contraparte	descripcion	fecha
1	1	1	RECARGA	100000.0	100000.0	NULL	Recarga	2026-10-02 21:26:13
2	2	1	PAGO	30000.0	70000.0	NULL	Pago en comercio	2026-10-02 21:26:13
3	3	2	RECARGA	500000.0	500000.0	NULL	Recarga	2026-10-02 21:26:13
4	4	2	PAGO	200000.0	300000.0	NULL	Pago a proveedor	2026-10-02 21:26:13
5	5	3	RECARGA	1000000.0	1000000.0	NULL	Capital de trabajo	2026-10-02 21:26:13
6	6	3	TRANSFERENCIA_ENVIADA	4500000.0	5500000.0	4	Pago de nomina	2026-10-02 21:26:13
7	7	4	TRANSFERENCIA_RECIBIDA	4500000.0	4500000.0	3	Pago de nomina	2026-10-02 21:26:13
8	8	3	TRANSFERENCIA_ENVIADA	3200000.0	2300000.0	5	Pago de nomina	2026-10-02 21:26:13
9	9	5	TRANSFERENCIA_RECIBIDA	3200000.0	3200000.0	3	Pago de nomina	2026-10-02 21:26:13
10	10	4	TRANSFERENCIA_ENVIADA	150000.5	4349999.5	1	Devolucion de prestamo	2026-10-02 21:26:13
11	11	1	TRANSFERENCIA_RECIBIDA	150000.5	280000.5	4	Devolucion de prestamo	2026-10-02 21:26:13
12	13	46	RECARGA	28000000.0	28000000.0	NULL	Recarga inicial	2026-09-01 08:00:00
13	14	38	RECARGA	42500000.0	42500000.0	NULL	Recarga inicial	2026-09-01 08:07:00
14	15	15	RECARGA	43000000.0	43000000.0	NULL	Recarga inicial	2026-09-01 08:14:00
15	16	25	RECARGA	36000000.0	36000000.0	NULL	Recarga inicial	2026-09-01 08:21:00
16	17	26	RECARGA	35000000.0	35000000.0	NULL	Recarga inicial	2026-09-01 08:28:00
17	18	45	RECARGA	39500000.0	39500000.0	NULL	Recarga inicial	2026-09-01 08:35:00
18	19	49	RECARGA	44000000.0	44000000.0	NULL	Recarga inicial	2026-09-01 08:42:00
19	20	33	RECARGA	44500000.0	44500000.0	NULL	Recarga inicial	2026-09-01 08:49:00
20	21	28	RECARGA	26000000.0	26000000.0	NULL	Recarga inicial	2026-09-01 08:56:00
21	22	18	RECARGA	3350000.0	3350000.0	NULL	Recarga inicial	2026-09-01 09:03:00
22	23	36	RECARGA	2800000.0	2800000.0	NULL	Recarga inicial	2026-09-01 09:10:00
23	24	39	RECARGA	4650000.0	4650000.0	NULL	Recarga inicial	2026-09-01 09:17:00
24	25	10	RECARGA	2150000.0	2150000.0	NULL	Recarga inicial	2026-09-01 09:24:00

Figura 17. Tabla transacciones en DB Browser for SQLite después de la carga: 590 registros, con las fechas originales de septiembre de 2026.

El identificador pasa de 11 a 13 porque el número 12 lo utilizó el usuario temporal de la prueba de borrado en cascada. Como `AUTOINCREMENT` no reutiliza identificadores, ese número no se vuelve a asignar.

```

1 SELECT tipo, COUNT(*) AS cantidad, ROUND(SUM(monto), 2) AS total
2 FROM transacciones GROUP BY tipo;

```

	tipo	cantidad	total
1	PAGO	249	113206105.8
2	RECARGA	127	807380000.0
3	TRANSFERENCIA_ENVIADA	107	80748865.18
4	TRANSFERENCIA_RECIBIDA	107	80748865.18

Figura 18. Consulta de verificación en la pestaña Execute SQL: cantidad y valor total de los movimientos por tipo.

RESULTADO DE LA VERIFICACIÓN

La base registra 249 pagos, 127 recargas y 107 transferencias. Las transferencias enviadas y las recibidas suman el mismo valor (80.748.865,18), lo que confirma que cada débito tiene su crédito correspondiente y que la carga masiva no creó ni eliminó dinero.

SQLite frente a PostgreSQL

El planteamiento de la actividad pregunta cuándo conviene migrar la persistencia de Quantum Core a un motor de mayor capacidad, como PostgreSQL, si la Fintech crece a millones de usuarios. SQLite es adecuado para la etapa actual, pero una Fintech en operación tiene exigencias que conviene anticipar. La comparación siguiente resume los criterios para tomar esa decisión.

CRITERIO	SQLITE	POSTGRESQL
Arquitectura	Biblioteca embebida; la base es un archivo local.	Servidor independiente con arquitectura cliente-servidor.
Concurrencia	Múltiples lectores, pero un solo escritor a la vez.	Múltiples escritores simultáneos mediante control de concurrencia multiversión (MVCC).
Seguridad	Depende de los permisos del archivo; no maneja usuarios.	Roles, privilegios por tabla y conexiones cifradas.
Valores monetarios	<code>REAL</code> de coma flotante; requiere redondeo en cada operación.	<code>NUMERIC(14, 2)</code> con precisión decimal exacta.
Crecimiento	Adecuado para aplicaciones locales, prototipos y volúmenes moderados.	Replicación, respaldos en caliente y alta disponibilidad.
Administración	Sin instalación ni configuración.	Requiere instalación, configuración y administración del servidor.

CRITERIO DE MIGRACIÓN

SQLite cubre el desarrollo y las pruebas de Quantum Core. Cuando el sistema atienda a varios usuarios que registren transacciones al mismo tiempo, o cuando maneje dinero real que exija precisión decimal exacta y control de acceso, corresponde migrar a PostgreSQL. El esquema diseñado se traslada casi sin cambios: `AUTOINCREMENT` pasa a columnas `GENERATED ALWAYS AS IDENTITY`, `REAL` a `NUMERIC(14, 2)` y los triggers a funciones en PL/pgSQL.

Conclusiones y oportunidades de mejora

La persistencia resuelve la volatilidad de Quantum Core. Los usuarios, las billeteras y los movimientos se conservan en `quantum_wallet.db`, aunque la aplicación se cierre, y el script reconstruye la base de forma idéntica en cada ejecución.

El modelo de clases se tradujo sin pérdida de significado. Cada relación del diagrama UML tiene su equivalente en el esquema: la composición se expresa con `ON DELETE CASCADE`, la agregación con una tabla intermedia que conserva al empleado y la herencia con una llave primaria compartida.

La integridad depende de la base de datos y no solo del código. Las llaves, las restricciones `CHECK` y los triggers rechazaron las ocho operaciones inválidas, incluidas las que se intentaron directamente con SQL, sin pasar por las funciones del script.

La carga masiva hereda la integridad del esquema. El importador no inserta datos por su cuenta: utiliza las mismas funciones y restricciones del sistema. Por esa razón detectó exactamente las 29 filas con errores entre 560 registros, conservó la conciliación de saldos y bloqueó la importación repetida del mismo archivo.

La elección de tipos tiene efecto directo sobre el negocio. El tipo `REAL` conservó los centavos de las transferencias; con `INTEGER`, los saldos habrían perdido la parte decimal.

Oportunidades de mejora

MEJORA	DESCRIPCIÓN
Montos en centavos	Almacenar los valores monetarios como enteros en centavos, o como <code>NUMERIC</code> en PostgreSQL, para eliminar el redondeo de coma flotante.
Validación previa de la carga	Ofrecer un modo de simulación que valide el archivo completo y reporte los errores antes de importar, para que el área operativa corrija el Excel en una sola revisión.
Capa de acceso a datos	Integrar las funciones del script en las clases del modelo, de modo que <code>Usuario</code> y <code>Wallet</code> lean y escriban directamente en la base.

REPOSITORIO

El código fuente, la base de datos, los scripts de generación e importación de Excel, el archivo de carga, el modelo entidad-relación (fuente `.drawio` y exportaciones PNG y SVG), el README y este informe se encuentran en github.com/DavisZulu/Semana_7_Act_1.

Referencias

- Castro, E. y de Pablo, C. (2013). *Desarrollo de bases de datos: casos prácticos*. RA-MA.
- DB Browser for SQLite. (2026). *DB Browser for SQLite* [Software]. <https://sqlitebrowser.org>
- Gazoni, E. y Clark, C. (2026). *openpyxl: A Python library to read/write Excel 2010 xlsx/xlsm files* (versión 3.1.5) [Software]. <https://openpyxl.readthedocs.io>
- Hueso, L. (2012). *Gestión de bases de datos* (2.^a ed.). RA-MA.
- Peláez Loaiza, S. (2026). *FUNDAMENTOS-DE-SOFTWARE* [Repositorio de código]. GitHub. <https://github.com/SimonP8/FUNDAMENTOS-DE-SOFTWARE>
- PostgreSQL Global Development Group. (2026). *PostgreSQL documentation*. <https://www.postgresql.org/docs/>
- Python Software Foundation. (2026a). *sqlite3: DB-API 2.0 interface for SQLite databases*. The Python Standard Library. <https://docs.python.org/3/library/sqlite3.html>
- Python Software Foundation. (2026b). *hashlib: Secure hashes and message digests*. The Python Standard Library. <https://docs.python.org/3/library/hashlib.html>
- Python Software Foundation. (2026c). *logging: Logging facility for Python*. The Python Standard Library. <https://docs.python.org/3/library/logging.html>
- SQLite. (2026a). *Datatypes in SQLite*. <https://www.sqlite.org/datatype3.html>
- SQLite. (2026b). *SQLite foreign key support*. <https://www.sqlite.org/foreignkeys.html>
- SQLite. (2026c). *SQLite autoincrement*. <https://www.sqlite.org/autoinc.html>
- W3Schools. (2026). *SQL CREATE TABLE statement*. https://www.w3schools.com/sql/sql_create_table.asp